

PKI-PQC 성능 및 비용 분석

(1) Agent 활용

<https://youtu.be/LGsfqOMhOK0>

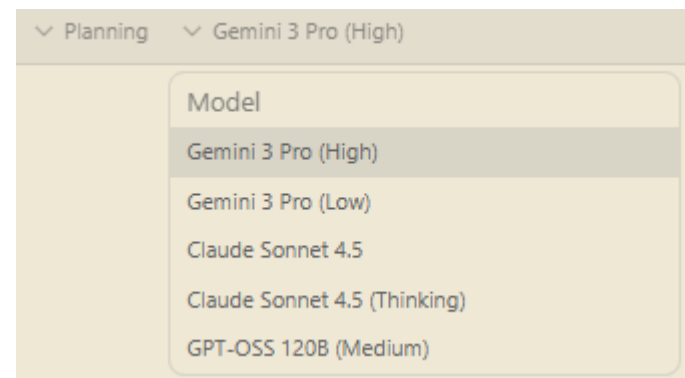
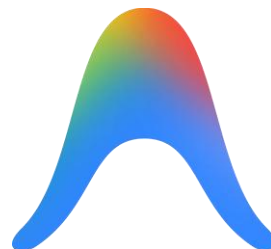
0. 프로젝트 구현 방법

- Cursor - Agent Mode를 활용
 - Claude Opus 4.5 모델 활용

- Claude Code를 설치해서 사용할 수 있지만, Cursor에서 더 쉽게 접근하여 사용할 수 있음

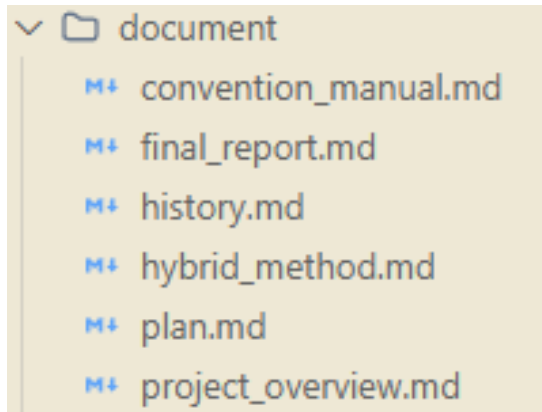


- Google Antigravity로도 구현 가능
 - 최근에 발표된 Google에서 개발한 Vscode 기반의 IDE
 - Cursor와 동일하게 동작
 - Gemini 3 Pro로 구현



0. 프로젝트 구현 방법 – Google Antigravity

- 프로젝트 구체화 및 Agent 구현 규칙 문서화
 - Document 디렉토리 내에 Plan, history, manual과 같은 마크다운(.md) 파일을 생성하고 Agent가 동작할 때 지속적으로 update 하거나 참고할 수 있도록 함.
- 프로젝트의 plan을 내가 원하는 대로 제대로 작성되도록 하는 것이 중요



프로젝트 히스토리 (Project History)
본 문서는 <code>convention_manual.md</code> 의 규칙에 따라 프로젝트의 진행 상황, 주요 변경 사항, 이슈 및 해결 과정을 기록한다.
Phase 0: Project Planning
- Status: 완료 (Completed) - Description: - 프로젝트 목표 및 범위 설정 (<code>project_overview.md</code>). - 하이브리드 인증서 기법 분석 (<code>hybrid_method.md</code>). - 구현 계획 수립 (<code>plan.md</code>). - 컨벤션 및 히스토리 관리 체계 수립 (<code>convention_manual.md</code>, <code>history.md</code>). - 주요 산출물: <code>sandwich_auth_test/document/project_overview.md</code> <code>sandwich_auth_test/document/plan.md</code> <code>sandwich_auth_test/document/convention_manual.md</code> <code>sandwich_auth_test/document/history.md</code>

구현 계획 (Implementation Plan)
본 문서는 <code>project_overview.md</code> 에 정의된 목표를 달성하기 위한 단계별 상세 구현 계획이다.
Phase 1: 환경 설정 및 기본 구조 설계 (Environment & Base Structure)
- 목표: 개발 환경 구축 및 공통 인터페이스 설계. - Tasks: - Python 가상환경 설정 및 의존성 설치 (<code>cryptography</code>, <code>liboqs-python</code>). - 디렉토리 구조 생성 (<code>src/</code>, <code>tests/</code>, <code>results/</code>). - 추상 기본 클래스(ABC) 정의: <code>CertificateAuthority</code> (Root/Intermediate 공통) <code>EndEntity</code> (Client) <code>Signer</code> / <code>Verifier</code> 인터페이스 (알고리즘 추상화) - Snapshot: <code>snapshots/phase_01_setup</code> 생성 및 <code>README.md</code> 작성.

PQC 및 하이브리드 인증서 성능 분석 프로젝트 (PQC & Hybrid Certificate Performance Analysis)

1. 프로젝트 개요 (Project Overview)

본 프로젝트는 기존 레거시(Legacy) 암호화 시스템에서 양자 내성 암호(PQC, Post-Quantum Cryptography)로의 전환을 대비하여, 다양한 인증서 발급 방식(Legacy, PQC, Hybrid)이 실제 PKI(Public Key Infrastructure) 환경에 미치는 영향을 분석하는 것을 목표로 한다.

특히, Root CA -> Intermediate CA -> Client로 이어지는 인증서 체인 구조에서 각 방식의 성능(속도, 크기) 및 **비용(검증 부하)**을 정량적으로 측정하고 비교 분석한다.

2. 프로젝트 목표 (Project Goals)

- 인증서 체인 구현: Root CA, Intermediate CA, Client 엔티티 및 인증서 발급/검증 프로세스를 코드로 구현한다.
- 다양한 알고리즘 적용: 모든 보안 강도(Security Level)별 파라미터 조합을 테스트하여 최적의 전환 시나리오를 도출한다.
- 다각적 성능 분석: 단일 클라이언트의 지연 시간(Latency)뿐만 아니라, 다중 클라이언트 환경에서의 처리량(Throughput) 및 리소스 사용량을 측정한다.

프로젝트 컨벤션 및 매뉴얼 (Project Convention & Manual)

본 문서는 `sandwich_auth_test` 프로젝트의 구현 및 문서화 과정에서 일관성을 유지하기 위해 정의된 규칙이다. 에이전트(AI)와 개발자는 본 규칙을 준수하여 결과물을 작성한다.

1. 문서 작성 규칙 (Documentation Rules)

1.1 언어 및 어조 (Language & Tone)

- 기본 언어: 한국어 (Korean)를 기본으로 작성한다.
- 기술 용어: 영문 원어를 병기하거나, 널리 쓰이는 한글 용어를 사용한다. (예: `Private Key` -> 개인키, `Handshake` -> 핸드셰이크)
- 어조: 명확하고 간결한 "해" 또는 "한다"체를 사용하되, 문서의 성격에 따라 통일한다. (기술 문서는 "한다"체 권장)

1.2 마크다운 포맷 (Markdown Format)

- 제목: `#` (H1)은 문서 제목에만 사용하며, 본문은 `##` (H2)부터 시작한다.
- 강조: 중요한 키워드, 파일명, 변수명은 `Backtick``으로 감싼다. (예: `plan.md`, `cert_gen.py`)
- 리스트: 순서가 있는 경우 `1.`, 없는 경우 `•` 또는 `-`를 사용한다.

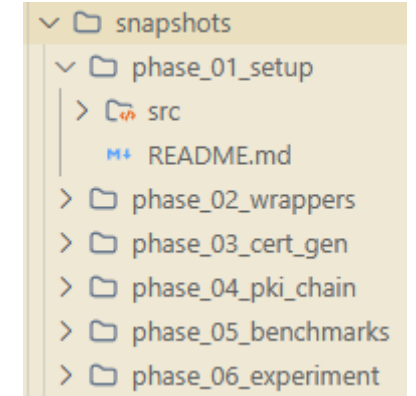
1.3 파일 명명 규칙 (File Naming)

- 문서: `snake_case.md` (예: `project_overview.md`, `hybrid_method.md`)
- 위차: 모든 프로젝트 문서는 `sandwich_auth_test/document/` 디렉토리에 위치한다.

0. 프로젝트 구현 방법 – Google Antigravity

- Google Antigravity – Gemini 3 Pro (High)

- 프로젝트 목표 설정 후 최종 프로젝트를 위한 Phase를 구분
 - 최종 프로젝트로 바로 완성될 경우 코드를 이해하는 데 어려울 것 같아서.
 - Phase 별로 Snapshot?을 통해 최종 코드와 분리하고 테스트를 할 수 있도록 구분함
 - 물론 Agent가 다 해줌



- 실제 동작하게 되면 Agent 자체에서도 Task와 implementation Plan을 계획

Project Planning Documentation

Goal Description

Update `convention_manual.md` and `plan.md` to incorporate the "Single `src` + Snapshot" workflow.

- **Convention:** Define the rule for developing in `src/` and archiving to `snapshots/phase_X/` with a `README.md`.
- **Plan:** Add a "Snapshot" step to the end of each Phase.

User Review Required

- Review the updated `convention_manual.md` and `plan.md` to ensure the snapshot workflow is clearly defined.

Implementation Plan

Tasks

- ☒ Analyze `hybrid_method.md` and user requirements
- ☒ Create Project Overview Document
 - ☒ Define Project Purpose
 - ☒ Define Project Goals (Performance/Cost Analysis)
 - ☒ Outline Methodology (Root -> Inter -> Client chain, Legacy/PQC/Hybrid comparison)
 - ☒ Summarize Hybrid Method from `hybrid_method.md`
- ☒ Update Project Overview with Expanded Scope
 - ☒ Add Multiple Client Scenarios (Concurrency/Load)
 - ☒ Add All Algorithm Parameters (Security Levels 1, 3, 5)
- ☒ Create Implementation Plan (`plan.md`)
- ☒ Create Convention Manual (`convention_manual.md`)
- ☒ Create Project History Log (`history.md`)
- ☒ Update `convention_manual.md` with Snapshot Strategy
- ☒ Update `plan.md` with Snapshot Steps

Task

0. 프로젝트 구현 방법 – Google Antigravity

- 결과 리포트까지 작성하고, 측정 결과도 CSV나 JSON으로 제공
- 다만, PQC 알고리즘이 제대로 동작하지 않아서 Mock로 가짜 구현체로 동작하도록 구현됨.
 - 제대로된 API를 안내하거나 중간에 Agent가 물어보는 질문에서 내 의견을 전달해야함.
 - Auto로 동작하면 그냥 쪽 진행될 수 있음
- Google의 Antigravity의 장점은 빠르고, agent 채팅 내역이 클라우드로 저장되는 것 그리고 Google.
 - 두 개 정도의 다른 프로젝트를 진행했을 때, 사용량 제한으로 4시간 정도 쉬는 시간

4.3 부하 테스트 (Throughput - Scenario B)

- 조건: 10 Concurrent Clients, 3초 지속.
- 결과 (TPS):

Algorithm	TPS	vs RSA	분석
ECDSA P-256	~2828	3.0x	가장 높은 처리량
RSA-2048	~926	1.0x	기준
Hybrid (Composite)	~506	0.55x	데이터 크기 증가로 인한 약간의 저하
ML-DSA-44	~26	0.03x	PQC 연산 비용(Simulated)으로 인한 급감

• 결론:

- **Hybrid(Composite)**의 효율성: 과도기적 모델(Legacy Issuer)에서는 Hybrid 방식이 순수 PQC 방식 보다 월등히 높은 처리량을 보입니다. 이는 PQC 서명 연산을 피하고, 단순히 PQC 키를 운반(Carrier) 하는 역할만 수행하기 때문입니다.
- **PQC** 도입 시 주의: 순수 PQC로 전환 시(ML-DSA), 소프트웨어 처리량은 현재의 3% 수준으로 떨어질 수 있습니다. 이는 **하드웨어 가속기** 없이는 대규모 트래픽 처리가 불가능함을 시사합니다.

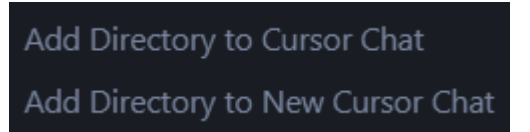
5. 결론 및 제언 (Conclusion)

1. **Hybrid 전략의 유효성**: Composite 방식은 인증서 크기가 2.5배 증가하지만, Legacy CA 인프라를 활용할 경우 처리량 저하를 최소화(~55% 수준 유지)하면서 PQC 키를 배포할 수 있는 효과적인 과도기 전략입니다.
2. **네트워크 대역폭**: 인증서 크기 증가(2.5배)는 네트워크 병목을 유발할 수 있으므로, **Falcon**과 같은 경량 알고리즘이나 **Related** 인증서(필요 시 다운로드) 방식 고려가 필요합니다.
3. **하드웨어 가속**: PQC 서명/검증의 높은 비용을 감당하기 위해 HSM 등 전용 하드웨어 도입이 필수적입니다.

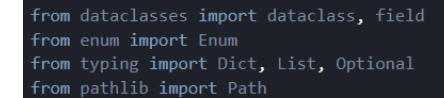
0. 프로젝트 구현 방법 – Cursor Claude Opus 4.5

• Cursor의 장점?

- 많은 모델이 있어서 여러 모델을 사용해서 비교해 볼 수 있음
- 생각보다 많은 사용량을 제공함
 - Claude Opus를 Claude Code로 사용하면 더 적은 사용량을 제공할 것 같은 느낌
 - 5시간 동안 한시간에 두 번 정도 진행하도록 해도 사용량 제한이 안 걸렸음.
- 아직까지는 다른 IDE 보다 훨씬 사용자 친화적으로 AI를 사용하기 좋음
 - 코드 작성중에 바로 명령으로 코드를 생성하는 것
 - 채팅창에 코드나 디렉토리를 지정하기



Add Directory to Cursor Chat
Add Directory to New Cursor Chat



```
from dataclasses import dataclass, field
from enum import Enum
from typing import Dict, List, Optional
from pathlib import Path
```

Edit selected code

Auto ~

Edit Selection ~

• Cursor의 단점?

- 채팅이 작업 공간에 종속?됨
 - A컴퓨터에서 작업하다가 B컴퓨터에서 작업하면 이전의 채팅 내용을 확인할 수 없음
 - X 디렉토리를 열어서 작업하다가 Y 디렉토리를 열어서 작업하면 채팅 내용을 확인할 수 없음
- Opus 모델 문제일수도 있으나, Google Antigravity 보다 체감상 3배 느림

0. 프로젝트 구현 방법 – Cursor Claude Opus 4.5

- Cursor Claude Opus 4.5에서도 동일하게 프로젝트 설명하고 Document를 생성
 - Gemini 3 Pro 보다 Opus 4.5가 문서 작성은 더 괜찮음

1. 프로젝트 규칙 개요

본 문서는 PKI PQC 성능 분석 프로젝트의 일관된 진행을 위한 규칙을 정의합니다.

2. 디렉토리 구조 (Directory Structure)

```
PKI_PQC/
├── document/
│   ├── project_rules.md    # 모든 문서 관리
│   ├── project_plan.md    # 프로젝트 규칙 (본 문서)
│   ├── history.md         # 프로젝트 진행 히스토리
│   ├── hybrid_method.md   # 하이브리드 기법 명세
│   └── results/           # 분석 결과 문서
├── phases/
│   ├── phase1_foundation/ # Phase 1: 기반 구축
│   │   ├── README.md     # Phase 설명 및 실행 방법
│   │   ├── src/           # 해당 Phase 소스 코드
│   │   └── tests/         # 해당 Phase 테스트 코드
│   ├── phase2_legacy/     # Phase 2: 레거시 알고리즘
│   ├── phase3_pqc/        # Phase 3: PQC 알고리즘
│   ├── phase4_hybrid/     # Phase 4: 하이브리드 기법
│   ├── phase5_benchmark/  # Phase 5: 벤치마크
│   ├── phase6_loadtest/   # Phase 6: 부하 테스트
│   └── phase7_analysis/   # Phase 7: 분석 및 문서화
```

history.md

프로젝트 진행 히스토리를 기록하는 문서입니다. 최신 항목이 상단에 위치합니다.

[2025-11-30] 최종 보고서 업데이트 - 네트워크 결과 통합

작업 내용

- FINAL_REPORT.md에 네트워크 시뮬레이션 결과 통합
- 레거시, PQC, 하이브리드 종합 네트워크 테스트 추가
- 네트워크 환경별 권장사항 추가
- 최종 권장 구성 정리

추가된 내용

- Section 5: 네트워크 시뮬레이션 결과 (전체)
- Section 2.4: 네트워크 용어 정의
- Section 7.3: 네트워크 환경별 최적화 전략
- 부록 C: 네트워크 시뮬레이션 조건

1.3.4 알고리즘 요약

전체 알고리즘 목록 (11개)

[Legacy - 6개]

- RSA: RSA-2048, RSA-3072, RSA-4096
- ECDSA: ECDSA-P256, ECDSA-P384, ECDSA-P521

[PQC - 5개]

- ML-DSA (Dilithium): ML-DSA-44, ML-DSA-65, ML-DSA-87
- Falcon: Falcon-512, Falcon-1024

```
document
├── history.md
├── hybrid_method.md
├── project_plan.md
└── project_rules.md
```

3.1 성능 순위

체인 생성 시간 (빠른 순)

1. 🏆 ECDSA-P256 + ML-DSA-44: ~1ms
2. 🏆 ECDSA-P256 + Falcon-512: ~20ms
3. 🏆 RSA-3072 + ML-DSA-44: ~430ms

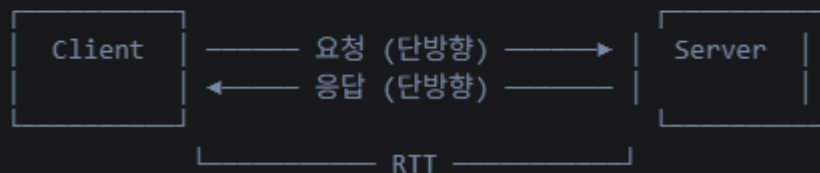
체인 크기 (작은 순)

1. 🏆 ECDSA-P256 + Falcon-512: ~12KB
2. 🏆 ECDSA-P256 + ML-DSA-44: ~25KB
3. 🏆 RSA-3072 + ML-DSA-44: ~29KB

2.4 네트워크 용어

RTT (Round-Trip Time)

클라이언트에서 서버로 요청을 보내고 응답을 받기까지의 왕복 시간



D. 참고 자료

- NIST Post-Quantum Cryptography Standardization
- RFC 5280 (X.509 PKI)
- liboqs Documentation
- Python cryptography Library

0. 프로젝트 구현 방법 – Cursor Claude Opus 4.5

프로젝트 구조	
PKI_PQC/	
├─ README.md	# 이 파일
├─ document/	
│ └─ project_plan.md	# 프로젝트 계획
│ └─ project_rules.md	# 프로젝트 규칙
│ └─ hybrid_method.md	# 하이브리드 기법 명세
│ └─ history.md	# 진행 히스토리
└─ phases/	
└─ phase1_foundation/	# 기반 구축
└─ phase2_legacy/	# RSA, ECDSA 구현
└─ phase3_pqc/	# ML-DSA, Falcon 구현
└─ phase4_hybrid/	# 4가지 하이브리드 기법
└─ phase5_benchmark/	# 성능 벤치마크
└─ phase6_loadtest/	# 부하 테스트
└─ phase7_analysis/	# 최종 분석 보고서
└─ phase8_network/	# 네트워크 시뮬레이션

분석 대상	
알고리즘	
유형	알고리즘
Legacy	RSA-2048/3072/4096, ECDSA-P256/P384/P521
PQC	ML-DSA-44/65/87, Falcon-512/1024
하이브리드 기법	
기법	설명
Composite	단일 인증서에 두 알고리즘 결합
Chameleon	Legacy 기본 + PQC 확장 필드
Catalyst	Alternative Public Key 방식
Related	분리된 두 인증서 상호 참조

테스트 현황			
Phase	내용	테스트	상태
1	기반 구축	87개	✓
2	Legacy 알고리즘	90개	✓
3	PQC 알고리즘	81개	✓
4	하이브리드 기법	53개	✓
5	벤치마크	20개	✓
6	부하 테스트	17개	✓
7	분석 및 문서화	-	✓
8	네트워크 시뮬레이션	16개	✓
총계		364개	✓

감 사 합 니 다